

Práctica 2. Conceptos Avanzados sobre Bases de Datos Relacionales

Versión 2.1 - 02/10/2026

Introducción

En esta práctica se profundiza en el uso de bases de datos (BBDD) relacionales usando PostgreSQL. Mediante procedimientos almacenados y triggers, el estudiante aprenderá a gestionar la persistencia y a trasladar parte de la lógica de negocio hacia la propia base de datos, reduciendo así la dependencia del código externo y mejorando la consistencia de las operaciones. Además, se llevará a cabo la optimización de queries.

Objetivos docentes

Tras la realización de esta práctica, el estudiante debe ser capaz de:

- Desplegar un contenedor Docker con la base de datos en PostgreSQL para su uso.
- Entender e implementar procedimientos almacenados.
- Entender y manejar los triggers en una base de datos.
- Identificar posibles acciones que pueden mejorar el rendimiento de una consulta.
- Acceder a una base de datos desde Python usando SQLAlchemy.

Tecnologías a utilizar

En esta práctica, se utilizará:

- **Quart** como framework ASGI (Asynchronous Server Gateway Interface).
- **Docker** como sistema de contenedores para distribuir los servicios.
- **PostgreSQL** como sistema gestor de bases de datos (SGBD). Se desplegará en un contenedor Docker y se arrancará mediante el fichero `docker-compose.yml` correspondiente. Se debe usar una imagen oficial de PostgreSQL, que cree el usuario `alumnodb` con contraseña `1234` y la base de datos `si1`.
- **SQLAlchemy** (librería Python). Se utilizará como interfaz de acceso a la base de datos en PostgreSQL desde la API REST.

En esta práctica, como en la anterior, todas las tecnologías se desplegarán en contenedores, por lo que debe programarse un único fichero `docker-compose.yml` para levantar y detener todos los contenedores y servicios necesarios.

Tareas a realizar

Se trata de implementar una pequeña plataforma de distribución de contenidos audiovisuales (un videoclub de películas), donde los usuarios se registrarán y podrán consultar el catálogo de películas, recargar el saldo de su cuenta, comprar contenidos y realizar valoraciones. Los datos persistirán en una base de datos.

1. Diseño y carga de la base de datos

El diseño de la base de datos es libre, y cada equipo de estudiantes deberá implementarla según su criterio. No obstante, el diseño debe cumplir con los requisitos que se enumeran a continuación.

1.1. Requisitos

El sistema deberá cumplir con los siguientes requisitos mínimos:

- **Gestión de usuarios.** El sistema deberá permitir gestionar diferentes usuarios que interactúen con la aplicación. Los usuarios deberán poder consultar información y realizar determinadas operaciones sobre las películas del videoclub.
- **Catálogo de películas.** El sistema deberá gestionar todo lo relacionado con las películas, actores y directores. También permitirá realizar diferentes consultas sobre estos elementos.
- **Recursos y disponibilidad.** El sistema deberá gestionar la compra de películas por parte de usuarios. Por tanto, deberá determinar si una película está disponible, cuántas unidades existen, etc., es decir, llevar un stock. Las compras de películas se realizarán mediante la metáfora del *carrito de la compra*. A la hora de pagar, hay que tener en cuenta aspectos como el saldo disponible del usuario, la reducción del stock, etc.

Se debe realizar un diseño formal de la base de datos mediante un diagrama E-R (Entidad Relación). A partir de esta base de datos, se debe construir la siguiente funcionalidad:

- Gestión de los recursos y disponibilidad mediante **triggers**. Por ejemplo, cada vez que se añada o elimine un producto del carrito, deberá actualizarse el stock, el precio final del carrito y/o cualquier otra acción necesaria.
- Queremos evitar problemas de sincronización con el usuario. Por ejemplo, cada vez que se pague un carrito, queremos que su coste se descuente automáticamente del saldo del cliente mediante **triggers**.

- Las consultas más complejas deben implementarse construyendo **procedimientos almacenados** (SPs). Por ejemplo, para el cálculo de la valoración media de una película se calculará el valor promedio de las puntuaciones usando SPs.

Para realizar estas implementaciones, se deben crear dos scripts SQL:

- **schema.sql**: con todas las sentencias DDL (Data Definition Language) necesarias para crear la estructura completa de la base de datos con los elementos comentados.
- **populate.sql**: con todas las sentencias DML (Data Manipulation Language) necesarias para insertar datos ficticios que servirán como punto de partida para la aplicación.

Posteriormente, se debe configurar el contenedor de la base de datos para que estos scripts se ejecuten automáticamente en el arranque, de modo que la primera vez que se levante el contenedor:

- Se cree la base de datos completa.
- Se carguen los datos iniciales.
- Los datos persistan en un volumen externo para que no se pierdan al reiniciar el contenedor.

2. Construcción de la API

En esta práctica se utilizará la librería SQLAlchemy para realizar queries contra el SGBD desde la API REST desarrollada en Python. Por tanto, se deberá implementar la funcionalidad necesaria para que los usuarios se autentifiquen y puedan consultar el catálogo de películas, recargar el saldo de su cuenta, comprar contenidos y realizar valoraciones. Para ello, igual que en la práctica anterior, usaremos un enfoque basado en microservicios.

2.1. Requisitos de implementación

El acceso a la base de datos debe realizarse exclusivamente mediante la librería SQLAlchemy, usando la interfaz Core (<https://docs.sqlalchemy.org/core>), que ofrece una capa de abstracción más cercana al lenguaje SQL, permitiendo construir y ejecutar sentencias SQL de forma explícita.

Dado que la API se publicará y ejecutará con Quart, que utiliza un modelo asíncrono de procesamiento, se debe usar la extensión para asincronismo de SQLAlchemy (<https://docs.sqlalchemy.org/orm/extensions/asyncio.html>). Esto implica:

- Usar `async/await`. En particular, todas las operaciones que accedan a la base de datos deben realizarse dentro de funciones declaradas como `async`, utilizando `await` cuando corresponda. Se debe mantener la coherencia del flujo asíncrono a lo largo del código.
- Utilizar las extensiones `asyncio` en la configuración del acceso a base de datos. El siguiente fragmento de código muestra una posible configuración para acceso asíncrono que puede servir de inspiración.

```
from quart import Quart
from sqlalchemy.orm import declarative_base, sessionmaker
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession

app = Quart(__name__)
DATABASE_URL = "postgresql+asyncpg://alumnodb:1234@localhost:9999/si1"
# --- Engine y sesión asíncronos ---
engine = create_async_engine(DATABASE_URL, echo=False)
async_session = sessionmaker(engine, expire_on_commit=False, class_=AsyncSession)

Base = declarative_base()
```

Finalmente, es importante señalar que pueden existir pequeñas diferencias entre versiones, por lo que es importante revisar la documentación correspondiente a la versión utilizada.

2.2. Funcionalidades mínimas de la API

Se crearán dos microservicios que se ejecutarán en dos contenedores independientes, programando los endpoints necesarios y separando las definiciones de la API según se indica a continuación. Los dos microservicios accederán a la misma base de datos.

2.2.1. API de usuarios

El servicio de gestión de usuarios almacenará el nombre de usuario y la contraseña cifrada. Tendrá además otros campos adicionales, como la nacionalidad (país) del usuario o el saldo de su cuenta, etc. La API de usuarios debe implementar las operaciones de crear y borrar un usuario, además de actualizar su saldo. También debe permitir hacer login y logout para actualizar el token de sesión. La API de usuarios se implementará en un fichero **user.py**.

2.2.2. API principal

La gestión y el acceso al videoclub se realizará usando también SQLAlchemy Core. Todas las sentencias SQL se escribirán manualmente y se ejecutarán usando el concepto de *prepared statements* (consultas parametrizadas con un diccionario de valores), utilizando el motor asíncrono configurado con `text()`, `execute()` y `await`. Esto aporta mayor seguridad, evitando ataques de inyección SQL al separar los datos de la lógica de la consulta, además aporta mayor eficiencia, ya que las consultas pueden ser "precompiladas", reduciendo el tiempo de análisis y planificación por parte del motor de la base de datos. De esta forma, el código es también más mantenible.

La API (fichero **api.py**) debe incluir al menos los siguientes recursos y operaciones:

1. Gestión de recursos principales, soportando la creación y borrado de películas.
2. Obtener un listado de películas por algún criterio específico.
3. Funcionalidades relacionadas con el consumo de películas:

Sistemas Informáticos. Curso 2026/2027

Escuela Politécnica Superior. Universidad Autónoma de Madrid

- a. Añadir película al *carrito de la compra* (el pedido de películas a consumir no se gestionaría individualmente, sino en bloque a modo de carrito de la compra, donde se van añadiendo recursos al pedido y este se paga en conjunto).
 - b. Pagar el contenido del carrito de la compra.
 - c. Almacenar la fecha de pago de un pedido.
4. Consultas avanzadas:
- a. Permitir que un usuario vote una película (entre 0 y 10).
 - b. Consultar la puntuación media de los votos de una película.

Además, se debe implementar el siguiente endpoint que se utilizará más adelante:

- `.../ventas/<año>/<país>` que permita recuperar todas las ventas que se han hecho en un año dado para los clientes que pertenecen a un país determinado. Ejemplo: `/ventas/2026/Francia` devolverá el historial de compras del año 2026 de todos los usuarios con nacionalidad francesa.

En esta práctica, de forma parecida a la práctica anterior, la sesión de usuario se controlará mediante un token cifrado generado en el login. La comunicación será de tipo *stateless*, por lo que el token no se almacenará en la base de datos. Al igual que en la práctica anterior, el token debe enviarse cifrado en la cabecera de las peticiones mediante `Authorization: Bearer <token>`. Es necesario comprobar que el usuario está autenticado para ciertas operaciones.

Cada consulta deberá ejecutarse en un único `execute()`, no es válido lanzar varios `execute()` y luego combinar los resultados en python.

2.3. Cliente de prueba

Para evaluar estos dos microservicios, se debe crear un `client.py` que compruebe que se cumple con toda la funcionalidad implementada. El cliente deberá ser lo suficientemente completo como para probar todos los casos y los errores (tolerancia a fallos) correspondientes.

3. Optimización

En este último apartado vamos a estudiar aspectos de optimización. Para ello, vamos a utilizar la sentencia `EXPLAIN` sobre la consulta `.../ventas/<año>/<país>` para estudiar el plan de ejecución de la consulta. Con la sentencia `EXPLAIN` se puede identificar cómo mejorar el rendimiento de una consulta (por ejemplo, mediante la creación de un índice). Compara el resultado de `EXPLAIN` antes y después de hacer los cambios necesarios. Prueba distintas estrategias y discute los resultados.

Para ello, se debe entregar un script SQL llamado `optimizacion.sql` con todas las sentencias necesarias para optimizar la consulta comentada.

Entrega

Se entregará un único fichero ZIP (llamado `P2.zip`) que contenga el proyecto completo, con todos los ficheros necesarios para su ejecución. Esto incluye los ficheros fuente en Python, scripts, dependencias, fichero YAML, etc. Se puede incluir un fichero `README.txt` en la raíz con información e instrucciones básicas de despliegue y ejecución local. No se deben entregar entornos virtuales, cachés, bytecode ni metadatos del sistema operativo. Se debe utilizar una estructura de carpetas similar a la de la práctica anterior

El fichero `P2.zip` debe incluir la memoria de la práctica (`memoria.pdf`) con una descripción y capturas de la resolución de cada tarea exigida. Para esta práctica, la memoria debe incluir además una explicación del sistema y la toma de decisiones que han llevado a cabo sobre el diseño de la base de datos, incluyendo el modelo E-R de la base de datos diseñada. También se incluirá en la memoria la discusión sobre la optimización de la consulta del apartado anterior.

Planificación

Se recomienda la siguiente planificación para desarrollar la práctica:

Semana 1	Diseño de la base de datos, triggers y SPs
Semana 2	Programación de la API y microservicios
Resto de días hasta la entrega	Optimización de la consulta, memoria y entrega

Rúbrica de corrección

Se seguirá la siguiente rúbrica para la calificación de la práctica:

Concepto	Calificación Máxima
Diseño básico de la base de datos	1.5
Diseño avanzado: triggers y procedures	1.5
Microservicios y API implementados	3
Optimización	1
Cliente	1.5
Memoria	1.5
TOTAL	10

Sistemas Informáticos. Curso 2026/2027

Escuela Politécnica Superior. Universidad Autónoma de Madrid

En general, el código debe tener la calidad esperada y estar comentado. La tolerancia a fallos se debe garantizar, y se deben controlar los distintos códigos http en base a los servicios implementados. La memoria debe tener también el grado de completitud y la calidad esperados.

Se aplicará la penalización de 1 punto por día natural (o fracción) de retraso en la entrega. De la misma forma, se sancionará, según la normativa de la UAM, la copia o el uso indiscriminado de la IA.